Parcial Algoritmos y Programación II - 7541 Curso Calvo (1º instancia) 30 de octubre de 2015

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){
  int *A, *C, *F;
  int **B;
  char *D, *E;
  char G;
  int H;

  H = 65;
  G = 'D';
  A = &H;
  D = &G;
  B = &A;
  cout << (**B) <<  (*D) << (*A) << endl;

  C = new int[4];
  for (int i = 0; i < 4; i++) {
          C[i] = H + i;
  }

  F = C + 2;
  (*A) = 69;
  C[1] = H;
  (*F) = 67;
  (*C) = H;
  cout << C[0] << C[1] << C[2] << C[3] << endl;

  (**B) = C[0] + 2;
  A = C + 1;
  (**B) = H;
  C[0] = *A;
  (*F) = A[0] + 1;
  cout << C[2] << C[1] << C[0] << endl;

  E = (char*) A;
  C[1] = (**B) - 6;
  cout << (*D) << (*E) << G << endl;

  // liberar la memoria
  return 0;
}
```

2. Implementar para la clase Lista<T> con una estructura **simplemente enlazada** el siguiente método, indicando pre y post condiciones:

```
void agregar(T elemento, unsigned int posicionElemento);
```

3. Implementar el método `'filtrarActoresSecundarios'` de la clase `'Cinefilo'` a partir de las siguientes especificaciones:

```
class Cinefilo {

  public:

    /* post: procesa 'actores' y devuelve aquellos que NUNCA fueron protagonistas de una Película en 'peliculasAnalizadas',
     *       es decir que están en el reparto de alguna Película pero que nunca protagonizaron ninguna.
     *       Devuelve una nueva Lista con los Actores secundarios encontrados.
     */
    Lista<Actor*>* filtrarActoresSecundarios(Lista<Actor*>* actores, Lista<Pelicula*>* peliculasAnalizadas);
};
```

```
class Pelicula {

  public:

    /* post: Pelicula identificada por titulo y sin Actores asociados.
     */
    Pelicula(string titulo);

    ~Pelicula();

    /* post: devuelve el título de la Película.
     */
    string obtenerTitulo();

    /* post: devuelve los Actores que protagonizan la Película.
     */
    List<Actor*>* obtenerProtagonistas();

    /* post: devuelve TODOS los Actores que participaron de la Película,
     *       incluyendo los protagonistas.
     */
    Lista<Actor*>* obtenerReparto();
};
```

```
class Actor {

  public:

    /* post: Actor creado con el nombre indicado.
     */
    Actor(string nombre);

    ~Actor();

    /* post: nombre completo del Actor.
     */
    string obtenerNombre();

};
```

4. Diseñar la especificación e implementar el TDA **IntervaloNumerico**. Un Intervalo Numérico representa un subconjunto conexo de la recta Real, es decir, una parte de la recta entre dos valores dados. Debe proveer operaciones para:

- Crear el Intervalo indicando el límite inferior, el límite superior y si es abierto o cerrado.
- Devolver el límite inferior y el límite superior.
- Cambiar el límite inferior y cambiar el límite superior.
- Devolver la longitud del intervalo.
- Evaluar si un número dado como parámetro está o no dentro del intervalo.
- Evaluar si está incluído en otro intervalo dado como parámetro.
- Evaluar si incluye a otro intervalo dado como parámetro.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de cada uno de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas